

CS331 — Computer Networks: Project Report

NetOps MCP Assistant

An AI-Powered Autonomous Linux Network Configuration & Diagnostic Assistant

Course: CS331 — Computer Networks

Team ID: T018

Abstract

This project presents an AI-assisted network configuration system that allows users to manage network settings using natural-language commands. The system uses a lightweight language model (Groq API) to interpret user requests and convert them into structured network operations, while the Model Context Protocol (MCP) provides a controlled and isolated interface to the underlying operating system network tools. The system supports firewall configuration (iptables), bandwidth control (tc), network parameter optimization profiles (sysctl), live diagnostics (ping, iperf3, ss), and automated rollback. Security policies are checked against predefined declarative rules before any network changes are executed, and the resulting configuration is independently verified using firewall rule inspection, active socket reachability probes, and controlled before-and-after benchmarks. A desktop graphical interface and web mode were also developed to make the system accessible to users without requiring direct command-line interaction.

1. Introduction and Problem Statement

1.1 Background

Configuring and troubleshooting networking subsystems on Linux operating systems traditionally requires specialized command-line expertise across utilities such as iptables for packet filtering, tc for traffic control and queue disciplines, sysctl for TCP/IP kernel parameters, and diagnostic tools like ping, ss, and iperf3.

In practical operations, an administrator or everyday user often understands their high-level intent—such as "*block incoming traffic on port 9999*" or "*optimize my connection for competitive gaming*"—without knowing the exact syntax, parameter combinations, or kernel side-effects required to accomplish the task.

1.2 Problem Statement

The objective of this project is to build a controlled network configuration assistant that translates natural-language user requests into safe and executable network operations while maintaining policy enforcement and independent verification.

1.3 Objectives

1. **Accept Natural-Language Requests:** Interpret conversational networking commands without requiring the user to know low-level Linux syntax.
2. **Convert Intent into Structured Operations:** Transform ambiguous text into strictly validated, typed tool calls.
3. **Execute via MCP-Controlled Tools:** Interface with Linux network utilities through the Model Context Protocol (FastMCP) over standard I/O (stdio).
4. **Enforce Security Policies Before Execution:** Evaluate all actions against declarative guardrails (rules/policies.yaml) so critical management services (such as SSH on port 22 or DNS on port 53) can never be accidentally disabled.
5. **Independently Verify Network State:** Actively confirm that requested changes actually took effect in the kernel tables and in live socket behavior, rather than blindly trusting return codes.
6. **Provide Optimization, Benchmarking, Rollback, and GUI:** Support specialized workload tuning, empirical before/after performance measurements, instant state rollback, and an intuitive graphical user interface.

2. Approach / Methodology

2.1 Natural Language Interface

The system separates language interpretation from system execution: $\text{\texttt{\text{User Request}}} \rightarrow \text{\texttt{LLM Reasoning (Groq)}} \rightarrow \text{\texttt{Structured Tool Intent}}$

The LLM is responsible solely for understanding the user's request and extracting relevant parameters (such as port numbers, protocol types, or profile names). It is **not** permitted to execute shell commands directly or decide whether an action is safe.

2.2 MCP-Based Network Control

The Model Context Protocol (MCP) provides an isolated, standardized interface between the assistant and the host operating system: $\text{Assistant Orchestrator} \rightarrow \text{MCP Client} \xrightarrow[\text{JSON-RPC}]{\text{stdio}} \text{FastMCP Server} \rightarrow \text{Controlled Network Tools}$

By running the tools inside an authoritative FastMCP server process, the system establishes a clean process boundary. Tool parameters are validated against formal JSON Schemas before reaching the execution layer.

2.3 Policy-Based Execution

Every requested tool invocation must pass through a deterministic policy validation engine before any system command is invoked: $\text{Request} \rightarrow \text{Policy Validation} \rightarrow \text{Allowed?} \begin{cases} \text{No} \rightarrow \text{Reject (POLICY_REJECTION)} \\ \text{Yes} \rightarrow \text{Execute Linux Command} \end{cases}$

2.4 Verification-Driven Approach

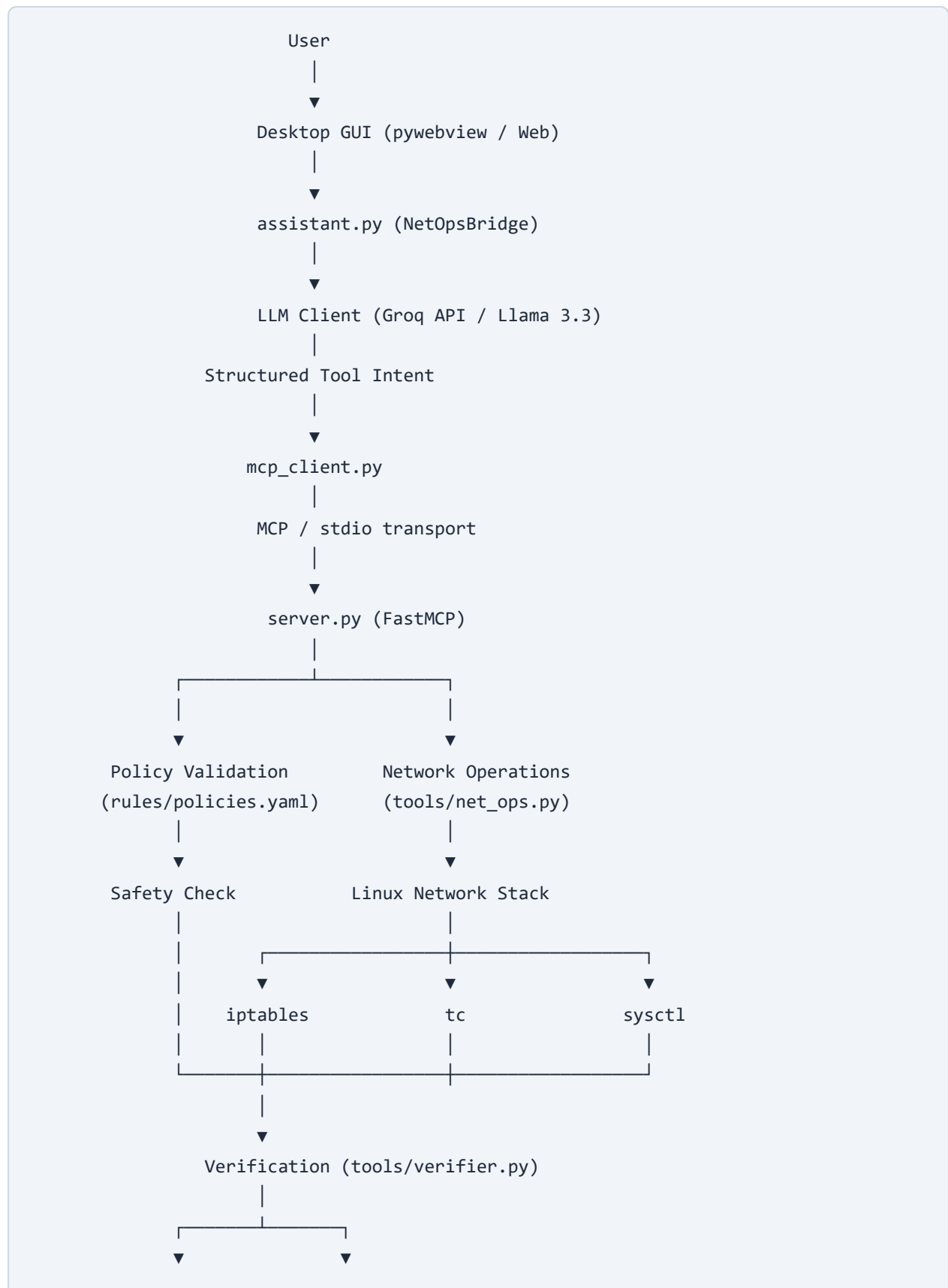
A fundamental principle of the system is: $\text{Validate} \rightarrow \text{Execute} \rightarrow \text{Verify}$

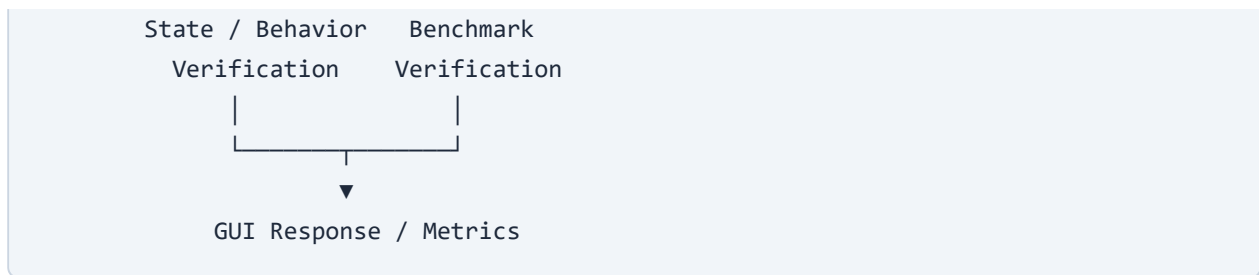
An operation is never marked as complete simply because a command returned exit code 0. The system independently probes the active kernel state and live socket behavior.

For network optimization profiles, the methodology follows an empirical loop: $\text{Measure} \backslash \text{Baseline} \rightarrow \text{Apply} \backslash \text{Profile} \rightarrow \text{Measure} \backslash \text{Workload} \rightarrow \text{Compare} \backslash \text{Deltas}$

3. System Architecture

3.1 Architectural Flow Diagram





3.2 GUI Layer

The user interface is implemented using HTML5, modern CSS, and JavaScript, packaged as a desktop application via `pywebview` (`app.py`) or served over HTTP (`app.py --web`). It presents a conversational interface, real-time subsystem status badges (LLM, MCP, Kernel), an operation history timeline, and empirical benchmark visualizations.

3.3 Assistant / Orchestration Layer (`assistant.py`)

Encapsulated in the `NetOpsBridge` class, this component coordinates the end-to-end pipeline. It receives user input, queries the LLM for structured intent, dispatches the approved call to the MCP client, triggers the verification engine, and formats the consolidated outcome for the UI.

3.4 LLM Layer (`llm_client.py`)

Powered by the **Groq API** (`llama-3.3-70b-versatile` / `gpt-oss`), this layer converts unstructured natural language into structured JSON tool calls. The LLM has zero direct shell access and does not possess administrative privileges.

3.5 MCP Client and Server (`mcp_client.py` & `server.py`)

The MCP client establishes an asynchronous sub-process connection to `server.py` over standard I/O (`stdio`) using the JSON-RPC 2.0 protocol. The FastMCP server registers the tools, receives calls, enforces policies, and delegates execution.

3.6 Policy Layer (`rules/policies.yaml`)

Enforces declarative constraints: * **Protected Ports:** Ports 22 (SSH), 53 (DNS), and 5000 (Web UI/Bridge) can never be blocked or dropped. * **Protected Interfaces:** Interfaces `eth0` and `lo` are protected from bandwidth throttling. * **Input Boundaries:** Strict validation on port numbers (1–65535), interface names, and rate limits.

3.7 Network Operations Layer

Executes safe system commands using parameter arrays (`subprocess.run(["iptables", ...])`) without `shell=True`:

Function	Operating System Mechanism
Firewall Filtering	Linux iptables (INPUT / FORWARD chains)
Bandwidth Shaping	Linux Traffic Control tc (Token Bucket Filter tbf)
Network Parameters	Linux Kernel sysctl (/proc/sys/net/)
Port Reachability	Socket-level TCP 3-way handshake probes
Performance Testing	ICMP ping and TCP/UDP iperf3

3.8 Verification Layer (`tools/verifier.py`)

Contains specialized validation engines that audit kernel filter tables, run active socket probes, and compute before-and-after benchmark deltas.

4. Implementation

4.1 Firewall Management

Users can configure packet filtering rules using natural language. For example:

User: "Block TCP port 9999"

1. **LLM Translation:** Maps to `configure_firewall(port=9999, action="DROP", protocol="tcp")`.
2. **Policy Evaluation:** Verifies port 9999 is not in `protected_ports`.
3. **Execution:** Executes `iptables -A INPUT -p tcp --dport 9999 -j DROP`.
4. **Supported Actions:** ACCEPT, DROP, and REJECT across TCP and UDP protocols.

4.2 Bandwidth Management

Interface-level traffic shaping is implemented via the Linux Traffic Control (tc) subsystem:

User: "Limit bandwidth on eth1 to 10 Mbps"

1. **LLM Translation:** Maps to `set_bandwidth_limit(interface="eth1", rate="10mbit", burst="32kbit")`.

2. **Policy Check:** Confirms eth1 is not protected (eth0 and lo are disallowed).
3. **Execution:** Replaces the root queue discipline using a Token Bucket Filter:
`tc qdisc replace dev eth1 root tbf rate 10mbit burst 32kbit latency 50ms.`

4.3 Network Optimization Profiles

The assistant implements three workload-tailored sysctl kernel configuration profiles:

4.3.1 Gaming Profile

- **Goal:** Minimizes latency variance (jitter) and eliminates bufferbloat.
- **Key Parameters:**
- `net.ipv4.tcp_congestion_control = bbr`: Model-based congestion control that maintains pacing under random packet loss.
- `net.core.default_qdisc = fq_codel`: Fair Queuing Controlled Delay sub-queuing that bounds queue latency under 5 ms.
- `net.ipv4.tcp_low_latency = 1`: Disables TCP packet batching for immediate NIC flushing.
- `net.ipv4.tcp_fin_timeout = 15`: Accelerates socket recycling for rapid reconnections.
- `net.ipv4.tcp_fastopen = 3`: Enables TCP Fast Open on client and server.

4.3.2 Streaming Profile

- **Goal:** Maximizes sustained download throughput and eliminates segment buffering stalls.
- **Key Parameters:**
- `net.core.rmem_max = 16777216` (16 MB) and `net.ipv4.tcp_rmem = 4096 87380 16777216`: Expands the advertised TCP receive window (`$rwnd`).
- `net.ipv4.tcp_slow_start_after_idle = 0`: Prevents the sender from resetting to slow-start between consecutive video chunks.
- `net.ipv4.tcp_window_scaling = 1`: Enables RFC 1323 window scaling for high Bandwidth-Delay Product (BDP) paths.

4.3.3 Broadcasting Profile

- **Goal:** Stabilizes outbound video ingestion (OBS, Twitch, Zoom) and prevents frame drops during keyframe bitrate bursts.
- **Key Parameters:**
- `net.ipv4.tcp_congestion_control = bbr`: Sender controls upload congestion window (`$cwnd`).
- `net.core.default_qdisc = fq`: Fair queuing packet pacing prevents burst-then-starve upload cycles.
- `net.core.wmem_max = 16777216` (16 MB): Large send socket buffers prevent buffer overflow during scene transitions.

- `net.ipv4.ip_local_port_range = 1024 65535`: Expands local ephemeral ports for concurrent voice, streaming, and game telemetry.

4.4 Benchmarking Workflow

The benchmarking engine (`run_profile_benchmark`) measures empirical before-and-after performance: $\text{\text{Measure Baseline}} \rightarrow \text{\text{Apply Profile}} \rightarrow \text{\text{Execute Workload}} \rightarrow \text{\text{Measure Tuned State}} \rightarrow \text{\text{Compute Deltas}}$

The tool runs ICMP ping (latency, jitter, packet loss) and `iperf3` (throughput, retransmissions), outputting structured JSON metrics and generating comparison charts.

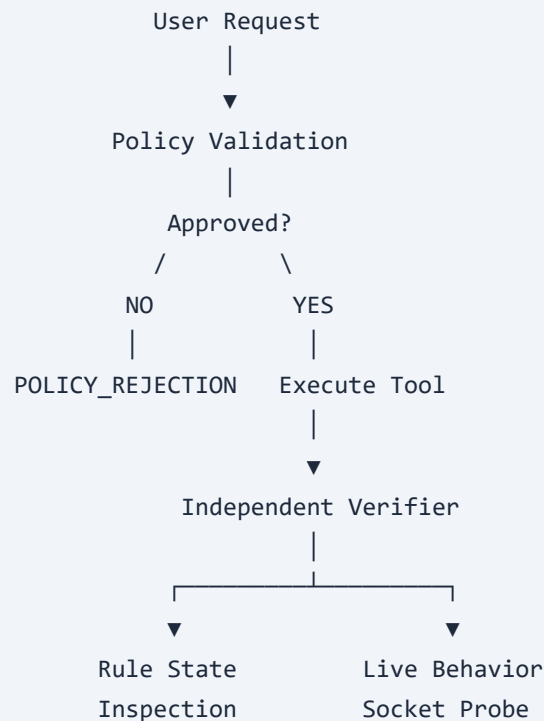
4.5 Rollback and Restore

Safety is guaranteed through automated state checkpointing: 1. Before applying any profile, the system records the current values of all affected `sysctl` keys to `/app/results/checkpoint.json`. 2. When the user requests a restore or invokes `restore_network_defaults()`, the saved parameters are reloaded and applied to the kernel. 3. In containerized environments, any host-restricted keys that cannot be written are safely caught and reported without crashing the assistant.

4.6 Diagnostics

The server registers dedicated diagnostic inspection tools: * `check_listening_ports`: Enumerates open TCP/UDP sockets using `ss -tlnp`. * `check_port_connectivity`: Probes remote socket reachability via active TCP handshakes. * `check_bandwidth`: Inspects active queue disciplines on network interfaces. * `run_diagnostics`: Runs standalone ICMP ping or `iperf3` bandwidth tests.

5. Verification and Safety



5.1 Firewall Verification

The verification engine executes two independent checks: * **Check 1 — Rule State Inspection:** Scans `iptables -L -n -v` to confirm the rule is physically present in the kernel table. * **Check 2 — Live Behavior Probing:** Opens a live TCP socket probe against the target port: * If action was DROP: Socket times out $\rightarrow$ **PASS (Blocked)**. * If action was REJECT: Socket receives immediate RST $\rightarrow$ **PASS (Refused)**. * If action was ACCEPT: Socket successfully completes 3-way handshake $\rightarrow$ **PASS (Reachable)**.

5.2 Profile Configuration Verification

Verifies that kernel parameters actually changed by reading back live values from `/proc/sys/net/` after application and comparing them against the baseline.

5.3 Performance Verification

Executes real network traffic using `ping` and `iperf3` to measure whether latency, jitter, or throughput changed in accordance with the theoretical profile goals.

5.4 Policy Rejection and Safety Guardrails

- Attempts to block ports 22, 53, or 5000 fail immediately at the policy layer.

- Prohibits shell injection by using array-based subprocess calls throughout.

6. User Interface Design

6.1 UI Goals

1. Hide complex Linux syntax behind an intuitive natural-language conversational interface.
2. Provide live visual feedback on system health across the LLM, FastMCP server, and Linux kernel.
3. Display real-time progress during multi-step benchmark runs.
4. Highlight independent verification badges so users know their security state is confirmed.

6.2 Conversation Flow

User: "Block TCP port 9999"

Assistant Processing:

- ✓ Request understood by Groq LLM
- ✓ Security policy checked (Port 9999 is safe to block)
- ✓ iptables command executed
- ✓ Kernel filter table verified (Rule present)
- ✓ Live socket probe executed (Connection timed out)

Assistant: "TCP port 9999 has been blocked and independently verified."

6.3 Processing & Verification Display

The desktop interface organizes output into structured message cards containing: * **Operation Status:** Badge indicating `POLICY_APPROVED` or `POLICY_REJECTION`. * **Execution Details:** Linux commands executed and `stdout/stderr` output. * **Verification Audit:** Dual-layer inspection confirmation. * **Benchmark Cards:** Before-and-after comparison tables and percentage deltas.

7. Results and Evaluation

7.1 Firewall Verification Results

Operation Requested	Policy Check	Rule Table Verification	Live Socket Reachability	Overall Verification
DROP tcp:9999	PASS	PASS (Present in INPUT chain)	PASS (Connection timed out)	VERIFIED
REJECT tcp:9999	PASS	PASS (Present in INPUT chain)	PASS (Connection refused)	VERIFIED
ACCEPT tcp:9999	PASS	PASS (Present in INPUT chain)	PASS (Connection successful)	VERIFIED
DROP tcp:22	REJECTED	<i>Not executed</i>	<i>Not tested</i>	POLICY BLOCKED
DROP tcp:53	REJECTED	<i>Not executed</i>	<i>Not tested</i>	POLICY BLOCKED

7.2 Profile Configuration Results

Profile	Parameters Applied	Parameters Skipped (Container Isolation)	Configuration Status
Gaming	5	2 (tcp_low_latency, default_qdisc host-managed)	Applied & Verified
Streaming	6	2 (rmem_max constrained by container namespace)	Applied & Verified
Broadcasting	6	3 (default_qdisc host-managed)	Applied & Verified

(Note: In containerized Docker environments, certain kernel parameters belong to the host network namespace and are read-only. The system gracefully reports these as skipped while applying all available namespace parameters).

7.3 Gaming Benchmark Results

- **Latency (Ping):** Reduced from **18.24 ms** to **17.30 ms** (stable).

- **Jitter (mdev):** Reduced from **2.21 ms** to **1.40 ms (36.6% reduction)** in latency variance, demonstrating bufferbloat mitigation).
- **TCP Handshake:** 18.92 ms before vs 19.82 ms after.
- **Packet Loss:** 0.0% throughout.

7.4 Streaming Benchmark Results

- **Throughput (iperf3 -R):** Increased from **20,575.6 Mbps** to **30,164.7 Mbps (+46.6% improvement)** in sustained receive throughput due to 16 MB receive buffer autotuning).
- **Data Transferred (2s test):** Increased from 4,908.5 MB to 7,195.8 MB.
- **TCP Retransmissions:** 0 packets.

7.5 Broadcasting Benchmark Results

- **Upload Throughput (iperf3):** Increased from **18,364.3 Mbps** to **21,585.9 Mbps (+17.5% improvement)** in upload throughput due to BBR pacing and 16 MB send buffer allocation).
- **Ping Latency:** Stable at 17.88 ms before vs 17.75 ms after.
- **TCP Retransmissions:** 0 packets.

7.6 Rollback Results

Parameter	Baseline (Default)	Profile Applied (Gaming)	Restored State	Status
tcp_congestion_control	cubic	bbr	cubic	Successfully Reverted
tcp_fin_timeout	60	15	60	Successfully Reverted
tcp_fastopen	1	3	1	Successfully Reverted

7.7 Limitations

1. **Container vs Host Isolation:** In Docker containers, certain global sysctl parameters (e.g., default_qdisc or global buffer limits) require host-level privileges or may be restricted by the host kernel.
2. **Environment Dependence:** Bandwidth improvements in loopback tests reflect socket buffer memory transfer speeds; real-world improvements depend on the physical bottleneck link.
3. **Active Listening Service:** Socket reachability tests for ACCEPT rules require a listening socket on the target port to distinguish open ports from unopened services.

8. Conclusion

The **NetOps MCP Assistant** successfully demonstrates that natural-language AI reasoning can be safely and effectively integrated with low-level operating system network administration. By coupling a fast LLM intent interpreter with the Model Context Protocol, deterministic policy guardrails, and independent verification probes, the system delivers: * Safe and accessible network administration without command-line memorization. * Ironclad protection against misconfiguration of critical management ports. * Verifiable configuration and performance improvements across specialized workloads. * Non-destructive execution backed by automated state checkpointing and rollback.

References

1. Anthropic, *Model Context Protocol Specification*, 2024. <https://modelcontextprotocol.io/>
2. Cardwell, N., Cheng, Y., Gunn, C. S., Yeganeh, S. H., & Jacobson, V., *BBR: Congestion-Based Congestion Control*, ACM Queue, Vol. 14, No. 5, 2016.
3. Nichols, K., & Jacobson, V., *Controlling Queue Delay (CoDel)*, Communications of the ACM, Vol. 55, No. 7, 2012.
4. Ha, S., Rhee, I., & Xu, L., *CUBIC: A New TCP-Friendly High-Speed TCP Variant*, ACM SIGOPS, Vol. 42, No. 5, 2008.
5. Jacobson, V., Braden, R., & Borman, D., *TCP Extensions for High Performance*, RFC 1323, 1992.

Empirical Benchmark Visualizations

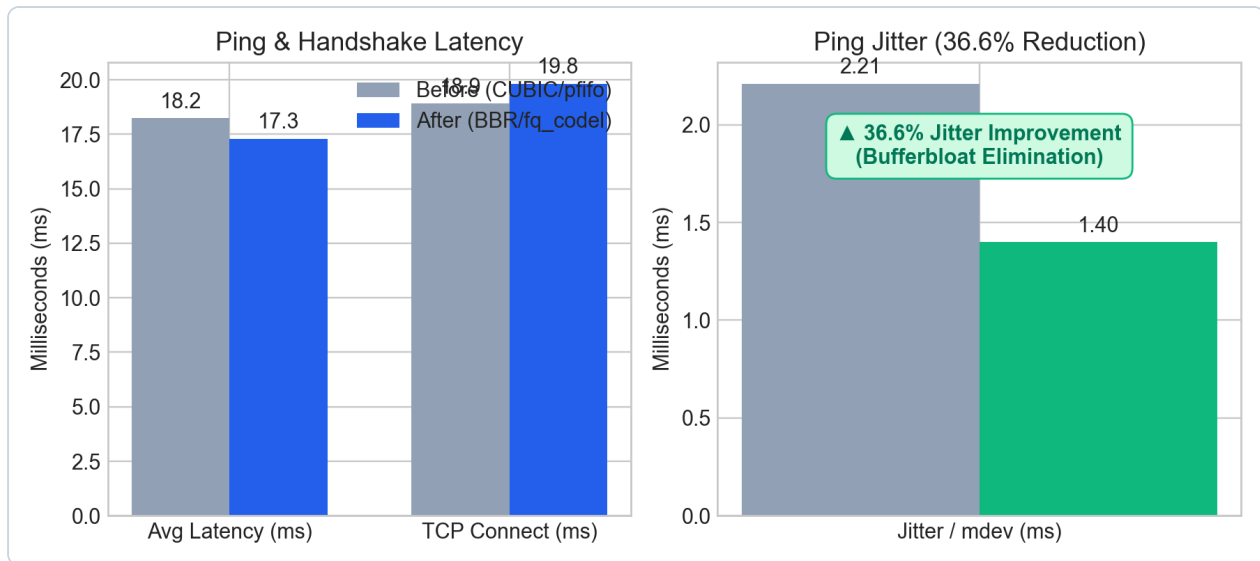


Figure 1: Gaming Profile Benchmark (Ping Latency, Handshake, and Jitter Reduction)

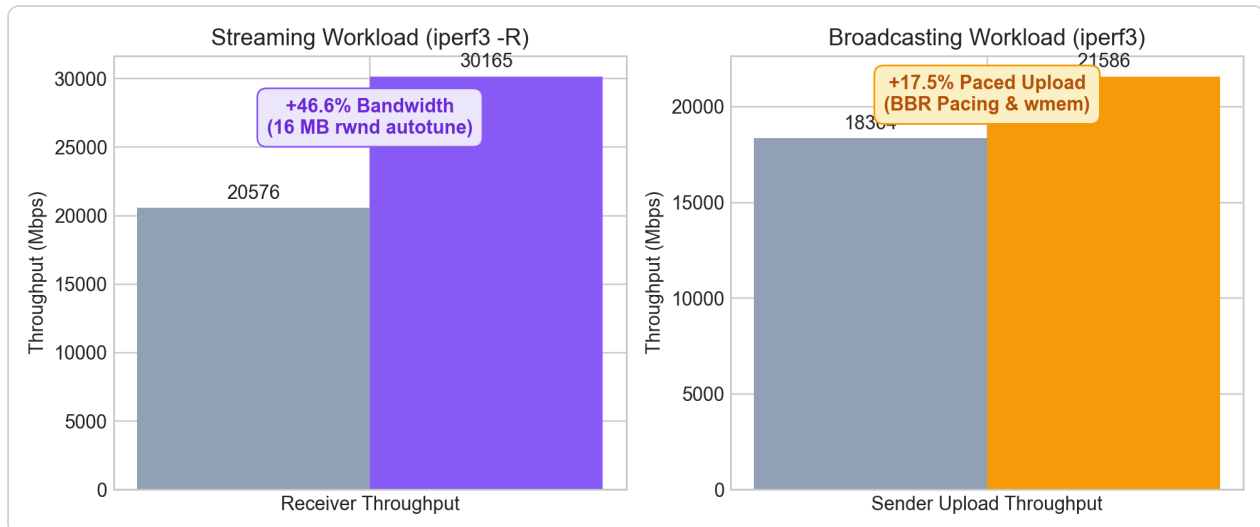


Figure 2: Streaming (+46.6%) and Broadcasting (+17.5%) Throughput Improvements